

Intelligent System for Autonomous Assistance in Elderly Care Homes



Project Report

Blanca Sánchez Santos (100496636)

Peng Yang (100472765)

Daniel Alonso González (100496374)

Marcos Gastón Rámila (100475134)

Miguel Fernández Lara (100473125)

1. Introduction	3
Project context and motivation	3
Objectives and scope	3
2. Data description	4
Sensor data overview	4
Variables and preprocessing	4
Assumptions and limitations	5
3. Phase 1: Exploration and unsupervised analysis	6
Data processing and exploratory analysis	6
Resident profiles and behavioral baselines	7
State definition and alert-based framework	7
Unsupervised detection of unusual behavior	7
Anomaly detection and sensor reliability	7
Phase 1 outcome	8
4. Phase 2: Supervised modeling and agent design	8
State labeling and validation	8
Supervised classification model	9
Alert system and severity levels	10
Heuristic rules	10
Decision-making agent logic and actions	11
5. Phase 3: Real-time operation	12
Streaming data pipeline	12
Real-time state prediction and alerts	13
Visual dashboard and robot monitoring	13
System logging and live metrics	14
6. System evaluation	14
Alert effectiveness and agent behavior	14
7. Results and use cases	15
Normal operation scenarios	15
Emergency detection	15
False alarm handling	15
8. Conclusions and future work	16
System limitations	16
Future improvements	16
Final conclusions	16
9. Appendices	17
Code structure	17
Dashboard video	17
Additional alert cases descriptions	17

1. Introduction

Project context and motivation

Elderly care residences face the challenge of providing continuous supervision while managing limited staff and increasing numbers of residents. Situations such as falls, unusually long absences from bed, or nighttime wandering can pose serious risks and may not always be detected immediately. With this context, our project focuses on a senior residence equipped with sensors placed in rooms and common areas to monitor daily activity patterns as well as wrist monitors for each resident.

The motivation behind this project is to improve safety and response times while supporting caregivers and enabling more proactive and efficient care.

Objectives and scope

The main objective of this project is to design and evaluate an intelligent system capable of monitoring elderly residents through sensor data and responding autonomously to potentially risky situations. The system aims to analyze continuous data streams, detect unusual behaviors that may indicate safety concerns, and make intelligent decisions about the appropriate response.

With our joint knowledge from diverse experiences and our degree, data science techniques are used to model normal activity patterns and identify anomalies such as possible falls, abnormal bed absences, or nighttime wandering. An autonomous agent will later on be responsible for interpreting ambiguous events together with other concrete systems and deciding whether to send an assistive robot or alert care staff. Our project focuses on the detection and decision-making components of the system, assuming the availability of sensor infrastructure and robotic platforms, but does not address hardware implementation or medical diagnosis.

2. Data description

Sensor data overview

- identification:
 - `resident_id`: Unique identifier for each resident
 - `timestamp`: Date and time of the recorded observation (per minute)
- wearable sensors:
 - `accelerometer_z`: Vertical acceleration, used to detect movement or falls
 - `pulse_bpm`: Heart rate in beats per minute
 - `wearable_location`: Location of the resident as reported by the wearable device (room, garden, rehabilitation...)
 - `wearable_alert`: Alert flag generated by the wearable
- radar-based monitoring:
 - `location_x`, `location_y`: Resident's position within the room area
 - `radar_event`: Movement or presence events detected by radar sensors
- environmental sensors:
 - `temperature_c`: Ambient temperature
 - `humidity_perc`: Relative humidity
 - `smoke_ppm`: Smoke concentration level
- others:
 - `bed_presence`: Indicates whether the resident is in bed
 - `door_state`: Open or closed state of the room door
 - `room_light`: Status of the room lighting

Variables and preprocessing

1. Variable Identification

The initial step involved categorizing the raw features into distinct types to guide our final preprocessing strategy. The following classifications were established:

- **Numerical Columns:** `resident_id`, `location_x`, `location_y`, `age`, `accelerometer_z`, `pulse_bpm`, `smoke_ppm`, `temperature_c`, and `humidity_perc`.
- **Boolean Columns:** `wearable_alert`, `bed_presence`, `door_state`, `room_light`, and a newly engineered feature `is_out_of_room`.
- **Categorical Columns:** `resident_state`, `radar_event`, `wearable_location`, `name`, and `condition`.
- **Datetime Columns:** The `timestamp` column.

2. Handling Missing Values

Missing data was addressed with specific strategies based on variable type:

- **Location Data (`location_x`, `location_y`):** Missing values in these columns were imputed with 0. This imputation served a dual purpose: it filled the gaps and implicitly

signaled that the resident was 'out of room', prompting the creation of the `is_out_of_room` boolean feature.

- **Other Numerical Columns:** These columns were found to have no missing values, hence no imputation was performed.
- **Categorical Columns:** These columns were also found to be complete, requiring no imputation.

3. Feature Engineering: `is_out_of_room`

A new boolean feature, `is_out_of_room`, was created to explicitly capture instances where a resident's location data was missing. This feature is True if either `location_x` or `location_y` was originally NaN (before imputation to 0), and False otherwise.

4. Categorical Encoding: One-Hot Encoding

To convert categorical variables into a numerical format suitable for machine learning models, One-Hot Encoding was applied. This process involved:

- Excluding the timestamp column, as it would be handled separately for time-series analysis if needed.
- Converting all identified categorical and boolean columns into numerical representations, creating new binary columns for each unique category. The `drop_first=True` argument was used to prevent multicollinearity.

After these preprocessing steps, the dataset (`df_encoded`) was transformed into a fully numerical format, ready for further analysis and model development

Assumptions and limitations

The nature of this project lead to many key assumptions and limitations:

- **Simulated Data:** The use of simulated data limits direct real-world generalizability and may not capture all complexities of actual elder care scenarios.
- **Limited Resident Pool:** The small sample of 20 residents may not be representative of a diverse elder population.
- **Missing Location Imputation:** Assuming missing location data means 'out of room' is a heuristic that might not always be accurate.
- **Fixed Thresholds:** The deterministic alert system relies on static physiological and environmental thresholds, which might not be optimal for all individual residents.
- **Rule-Based Logic:** The rigid rule-based system for alerts can be brittle and may misclassify novel or uncaptured events.
- **LLM Dependence:** The AI agent's (LLM) classification of ambiguous cases depends on its interpretation, context window size, and inherent biases.
- **Model Simplifications:** The neural network's architecture and hyperparameters are experimental choices, and treating data points as isolated sequences might not fully leverage complex temporal dynamics.

3. Phase 1: Exploration and unsupervised analysis

Phase 1 focused on exploring and structuring the unlabelled sensor data collected with the objective of understanding resident behavior, defining meaningful alert states, and detecting unusual or potentially risky situations. This phase established the analytical foundation for later supervised modeling and our agent-based decision making.

Data processing and exploratory analysis

The initial step involved cleaning and preprocessing a multimodal dataset composed of physiological, environmental, spatial, and categorical variables. Our exploratory data analysis revealed strong relationships between movement, location, and environmental conditions. Temporal patterns highlighted structured daily routines, including sleeping hours, meal times, and periods of outdoor activity, while spatial analysis of room coordinates allowed the identification of functional areas such as beds and bathrooms.

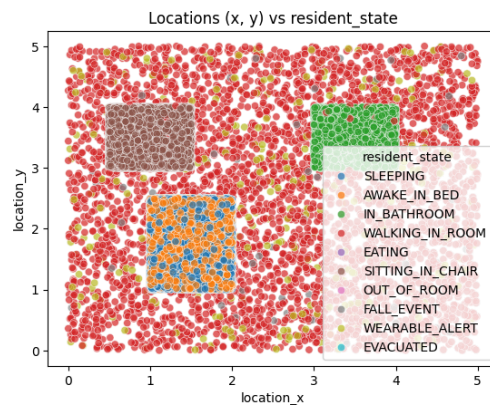


Figure 1: Locations by Resident State

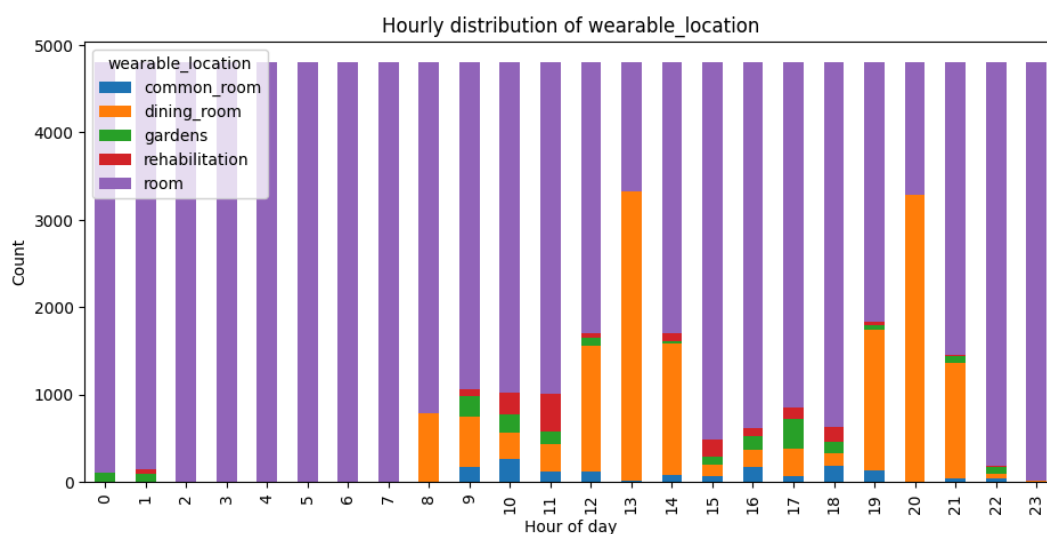


Figure 2: Hourly distribution of wearable location

In the first plot above (Fig. 1), one can observe the locations by resident state. There are 3 clear places where residents can be inside the room: a dining area, the bed and the bathroom. Outside of the resident's room, the location value is 0. Besides, in Fig. 2, one can observe the night hours (8h - 22h) and in what locations the residents spend their day.

Resident profiles and behavioral baselines

Individual resident profiles were obtained to summarize typical activity levels, heart rate ranges, environmental exposure, and location usage. These profiles gave us personalized behavioral baselines, highlighting the strong differences between residents. While most residents exhibited stable routines, some showed irregular behavior or sensor-related inconsistencies, which later aided our anomaly detection strategies.

State definition and alert-based framework

An initial attempt to define behavioral states was compared with the ground-truth states provided in the dataset, achieving partial agreement and revealing the ambiguity of activity-based labeling. To better align with the project's operational goals, a four-level alert-based triage system was introduced: Normal, Supervision, Risk, and Emergency. These levels were defined through interpretable logical rules combining physiological signals, movement patterns, and environmental conditions. This framework prioritizes rapid response to potentially dangerous situations such as falls, agitation, heat stress, or fire risk, independent of the resident's current activity.

Unsupervised detection of unusual behavior

Unsupervised methods were applied to identify deviations from normal routines without relying on labeled anomalies. Isolation Forest and K-Means clustering were used independently for each resident over 15-minute time windows. The resulting anomaly scores were combined into a fused score, and the top 5% most unusual windows per resident were flagged. Cross-model analysis showed strong agreement between both methods, increasing confidence in the detected anomalies. Most variability across residents came from data-driven models rather than from heuristic rules, which served primarily as an interpretability and prioritization layer (this would later be implemented in *Phase 2*).

Anomaly detection and sensor reliability

Additional anomaly detection analyses were conducted to identify both behavioral irregularities and sensor malfunctions. Rolling-window methods detected sharp physiological spikes but were limited in handling constant or noisy signals. Multivariate Isolation Forest models, with extensive feature engineering, proved effective in capturing complex interactions between physiological and environmental variables, identifying falls, abnormal vital signs, and faulty sensors. Modeling residents independently was key to achieving robust results.

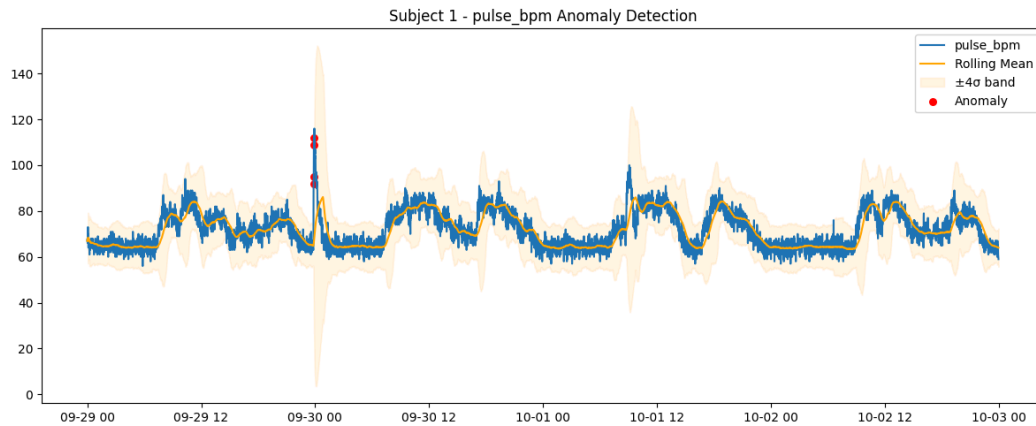


Figure 3: Pulse plot with rolling mean anomalies

In the plot above (Fig. 3) one can observe an example of a resident's pulse over time with a clear anomaly in the timestamp 9:30:00, which will be later taken into consideration to detect the alerts. A high change in pulse can mean an emergency

Phase 1 outcome

In conclusion, *Phase 1* successfully transformed raw sensor data into structured behavioral insights through our data exploration, resident profiling, alert definition, and unsupervised anomaly detection. The resulting behavioral baselines, alert framework, and anomaly scores provide a solid basis for *Phase 2*, where supervised models and intelligent agents used these insights for real-time monitoring and decision making.

4. Phase 2: Supervised modeling and agent design

State labeling and validation

To define the states, the residents have to be categorized depending on their vitals and conditions in every moment. Before having access to the actual states provided in the dataset, we created our own preliminary set of states.

Two different sets of states were developed, one related with the behavioral state of each resident which refers to resident activity and the other with the emergency level of each situation. Being the first set of states closer to the labels later provided to us.

For setting these states clusters were used. Nevertheless, the states tend to be highly similar that is why, later for splitting the data some heuristics rules were used and with them the category of 'Unknown'

In the table below (Fig. 4), there is the comparison between the predicted label and the real ones.

Predicted label	Real label
Alert	WEARABLE_ALERT
Sleeping	SLEEPING
Bathroom time	IN_BATHROOM
Eating	EATING
Go to the bathroom in the nights/morning	IN_BATHROOM
Getting dressed and ready for the day	AWAKE_IN_BED
Wake up in the middle of the night	AWAKE_IN_BED
Nap time	SLEEPING
Getting ready to go to bed	AWAKE_IN_BED
Time outside	OUT_OF_ROOM
Exercises	OUT_OF_ROOM
Time in the common room	OUT_OF_ROOM
Unknown	-

Figure 4: Comparison between real and designed states

Comparing both states we find out that out of 115200 entries, 67552 are equal. Meaning that 58% were correct. Maybe this may seem like a small number but the comparison previously made is not completely accurate for example, 'Getting ready to go to bed' that can be 'WALKING_IN_ROOM' and 'AWAKE_IN_BED'.

Supervised classification model

For the automation in the assignment of the previous states, multiple models were developed with techniques such as GridSearchCV or Gradient Boosting. Nonetheless, due to its ability to adapt to temporal series and its capacity to deal with noisy datasets, a simple LSTM neural network with only 1 layer has been selected as the final model. The total accuracy is a pretty high number, 0.98, which makes sense as we are aware that this dataset was artificially generated. A more complex or SOTA model like transformers were not used for this problem due to the nature of the information (not real data) and the lack of sufficient rows of information to train a fully competent model.

Alert system and severity levels

The main block of this project is based on deciding the alert system to be implemented. The use of an agent may, at first sight, be the optimal option to tackle the problem, since it provides high level reasoning and optimal classification results for the alert levels. However, the data flow we are dealing with in this problem, which consists of processing many patients every minute, does not support the idea of making a high amount of LLM calls per minute, which is computationally expensive and can show delays in the automatic flow of timestamps.

The key point in deciding the best approach is to understand how the problem is set. The data given contains unlabeled data that do not show the alert levels, and most of the columns are numeric. Therefore, carefully designed heuristic rules can sort out and classify the information with no problem. However, there can always be ambiguities, hard to classify samples and combinations of variables that may not be foreseen. Here is where an agent can take place: deciding how to proceed with unknown rows that present ambiguous information.

The alert system is composed of four color coded labels (green, yellow, orange and red) ordered by increasing emergency level.

Heuristic rules

The heuristic rules are the first triage level of the system. The definition of the deterministic rules is carefully developed to cover the maximum number of possible alerts. Each of the rules comes with a brief description and explanation of why the alert is triggered. The rules are designed to be interpretable, to provide safety guarantees for critical conditions and to be mapped to actions.

Firstly, the behavioral states are divided into normal states, which can be observed at any time of the day, and night-allowed, acceptable during night hours (defined 22:00-06:00). Moreover, the radar activity encoding is defined as absent (0), static presence (1), slight movement (2) and active movement (3). Finally, historical data of each patient has been defined to work as a baseline to prevent false positives.

The red rules cover the immediate dangerous conditions that require an urgent action and a caretaker must be sent in any case. These conditions include the 'EVACUATED' state, extreme pulse, confirmed falls ('FALL_EVENT'), environmental hazards or a sudden change in the accelerometer.

The orange rules also cover high risk situations but not as urgent as the red ones. Here we include the baseline conditions of each resident to prevent false positives and check the situation before initiating a red alert. These baselines cover the pulse and movement. Other orange alerts include residents being outside of the room during night hours or a wearable alert combined with a high movement.

In the case of the yellow rules, supervision is needed if triggered. They are usual situations that do not require urgent action, just monitoring with either a robot or caretaker. These

alerts cover when patients have unusual states at night, moderate baseline deviations or a prolonged midday sleeping that has to be checked.

Finally, the green rules contain the normal behaviours of residents that are truly confirmed and do not present any emergency or alert. These rules include normal sleeping with static radar and very low accelerator, walking normally, eating, etc. The green alerts do not need any intervention.

Decision-making agent logic and actions

As explained before, if no heuristic rule is matching the current event of a patient, it will be labeled as ambiguous and the system escalates to a decision agent. This is the final decision making of the pipeline. An agent provides a more contextualized decision to provide the classification of the alerts and understands ambiguous cases with coherence.

In this case, two interchangeable backends are used to operate with the alert system: an OpenAI model and a local LLM running on Ollama. The use of two models can lead to contrasting decisions and predictions. The Ollama-based model, integrated via LangChain, is open source and easy to implement, while the OpenAI model is subject to a limited quota of paid tokens.

As a first insight in the configuration choices to ensure predictability, we set the temperature to 0 (deterministic outputs), while not providing historical events to prevent overloading and set a strict JSON schema for the output.

The system prompt is defined to explain the agent's role and what it must do in each situation. It must assign exactly one alert level and provide the explanation and recommended action, while prioritizing safety.

The allowed action logic can be observed in the figure below (Fig. 5).

Alert Level	Allowed Actions
GREEN	NONE
YELLOW	SEND ROBOT/ SEND CAREGIVER
ORANGE	SEND CAREGIVER
RED	SEND CAREGIVER

Figure 5: Actions by alert level

However, one must note that the agent cannot handle large rows of numerical values since llm's, based on transformers, are prepared to handle text. Therefore, a transformation of the input to the agent is made. Instead of providing numerical values for each of the rows, semantic indicators are modelled to reduce noise and variability, improving generalization across residents and aligning with human reasoning patterns (Fig. 6).

Numerical Variables	Semantic Indicators
Heart Rate Status	LOW, NORMAL, HIGH
Motion Intensity	LOW, NORMAL, HIGH, SUDDEN
Environmental Context	LOW, NORMAL, HIGH / ELEVATED, DANGEROUS
Timestamp	NIGHT, MORNING, MEAL_TIME, NAP_TIME, EVENING

Figure 6: Semantic indicators for agent input

Besides implementing the semantic indicators, a priority system is designed to structure the importance of variables. The primary context contains the resident state, heart rate status and motion intensity, which are the core indicators of safety. Moreover, the secondary context is the rest of supporting sensor variables and environmental signals such as the radar activity, wearable alerts, room presence and environmental conditions. Finally, the tertiary context covers the vulnerability factors such as age and reduced mobility conditions.

Finally, after processing the data the agent is required to output a JSON object that contains the alert level, alert description, recommended action and immediate action (to observe the concise next step). This allows direct integration into the dashboard.

5. Phase 3: Real-time operation

Phase 3 represents the final step of our project, where all components developed in the previous phases are integrated into a single system operating under simulated real-time conditions. While *Phases 1* and *2* focused on data understanding, state definition, model training, and decision logic, *Phase 3* evaluates whether the full pipeline can function coherently when data arrives sequentially, without access to future information or ground-truth labels.

The objective proposed in this phase is to demonstrate that the system can process streaming data, predict resident states, generate alerts, make decisions, and present the results in an interpretable and continuous way.

Streaming data pipeline

To simulate real-time operation, the system uses a full-day dataset corresponding to a new day of observations, where no behavioral state labels are provided. The data is processed strictly in chronological order, treating each row as a new incoming sensor reading, similar to how data would arrive in a real deployment.

At each time step, the system processes the next incoming observation using the same preprocessing pipeline defined in *Phase 1*, including feature scaling, categorical encoding,

and engineered variables such as the out-of-room indicator. The pipeline is reused without retraining, ensuring consistency between training and real-time operation.

The streaming simulation advances through discrete time steps, or “ticks”, allowing the system to update its internal state incrementally. This prevents the use of future information and ensures that decisions rely only on current and past data, realistically emulating a continuous 24-hour monitoring process.

Real-time state prediction and alerts

Once new sensor data is processed, the system performs real-time state prediction using the supervised LSTM model developed during *Phase 2*. For each resident, the information is passed to the model to infer the current behavioral state. Since no ground-truth labels are available in *Phase 3*, all states used at this stage are predicted rather than known.

The predicted state is then combined with current sensor values and contextual information, such as time of day to evaluate the alert system. The heuristic alert rules defined earlier are applied at every time step, classifying the situation into one of several alert levels, ranging from normal behavior to high-risk or emergency situations.

In cases where the heuristic rules cannot confidently classify the situation, the observation is marked as ambiguous. These ambiguous cases are then passed to the decision-making agent, which uses a language model to reason over a structured summary of the situation and propose an alert level and action. This hybrid approach efficiently manages clear situations while reserving more complex reasoning for uncertain cases, reducing overall computational cost.

To sum up, this process demonstrates that the system can continuously predict states and generate alerts in real time, adapting dynamically to changes in resident behavior and sensor readings.

Visual dashboard and robot monitoring

To support real-time monitoring and interpretability, an interactive dashboard was implemented using Gradio. The dashboard provides both a global overview of the residence and a detailed view of individual residents, allowing caregivers to quickly assess the system’s current state and respond to alerts when necessary.

The home view displays all residents as color-coded indicators, where each color represents the current alert level. This design enables rapid identification of critical situations, as high-risk cases are immediately visible without requiring manual inspection of individual data streams.

Selecting a resident opens a detailed monitoring panel showing the most recent sensor values, including physiological signals, motion indicators, environmental measurements, and location-related features. These values reflect the real-time data processed by the system and provide immediate context for the predicted state and alert level.

The dashboard also includes time-series visualizations of key signals over the last hour, such as heart rate and accelerometer readings, together with a timeline of predicted behavioral states. These plots help explain how the resident's behavior has evolved over time and why a specific alert has been generated.

In addition, the system displays a structured log of recent decisions, including timestamps, alert levels, explanations, and the corresponding actions taken. Actions such as monitoring, dispatching an assistive robot, or escalating to human caregivers are clearly indicated. Although no physical robot movement is simulated, the dashboard explicitly shows when a robot intervention would be initiated.

System logging and live metrics

In parallel with real-time visualization, the system maintains a structured log of all decisions made during operation. At each time step, relevant information such as the timestamp, resident identifier, predicted state, alert level, explanation, and chosen action is recorded in a growing log structure.

This logging mechanism allows the system's behavior to be inspected retrospectively and supports basic monitoring of live metrics. By observing the accumulated logs, it is possible to assess how frequently alerts occur, how often ambiguous situations arise, and which actions are most commonly triggered. This information is essential for evaluating system stability and detecting potential issues such as excessive false alarms or over-reliance on escalation.

6. System evaluation

The system is tested with an unlabeled dataset that contains the information of the patients during 03/10/2025, from time 00:00:00 until 23:59:00. In total, there are 28800 rows of data for all patients together.

Alert effectiveness and agent behavior

When testing the system with the dataset that contains the first three days (labeled one), which contains 115200 rows, the number of ambiguous cases with the heuristic rules defined is 25827, a 22% of the total rows. This means that the use of the agent is highly reduced while allowing the most bizarre cases to be processed by the agent.

However, in the case of the alert system with the test data, we can observe 15560 labeled rows with the heuristic rules while there are 13240 ambiguous rows. This means that 46% of the rows are being recognized as ambiguous. This increase highlights the sensitivity of the agent to classifier uncertainty rather than system instability. This means that either the data on this day is chaotic or that the state prediction from the classifier is not working properly. When observing the root of the problem, the state system is sometimes classifying wrongly, as explained in the previous section. Nevertheless, passing more rows to the agent is not a major problem, it simply increases the number of processed cases while improving decision coherence. The limitation comes when the agent takes into account the wrongly classified resident states.

Overall, it performs in a very accurate way and provides very well developed alerts and reasonings. The agent always acts as it is asked to. One important thing to note is that the agent almost never sends a robot to check on the residents when there is a yellow alert. It has been discovered that since we are asking the agent to always prioritize safety, it will always send a caretaker over a robot. When trying to change this in the system prompt, it actually triggered more false positives and the final decision was to leave it as it is.

7. Results and use cases

The examples of the results described below are from the testing dataset, corresponding to the last day, which has no labeled states.

Normal operation scenarios

Most of the system output is either GREEN or YELLOW alerts, which represent normality or not urgent monitoring.

For example, at 00:29:00, Adela Arco-Llano is walking in the room. This corresponds to a night period and therefore is classified as an ambiguous state for the agent to understand the nature of the situation. Since the agent checks that the vitals are correct and no danger is present, the alert is assigned green, with the reason: 'Resident walking normally at night'.

Emergency detection

On this day, one can observe a smoke outreach around 16:16:00, for the resident Cristian Cazorla Cazorla, and the system is capable of successfully identifying and describing the problem. The immediate action is correct since it sends an urgent caretaker.

Another emergency present can be observed at 17:40:00, where Graciela Zamorano Vergara is assigned a RED label due to a fall event and also high smoke concentration.

Finally, another example is that at 11:38:00, there are two residents that are assigned EVACUATED state, therefore the alert given is RED. They seem to have normal vitals but the EVACUATED state is treated instantly as a RED alert and a caretaker must take action.

To observe more alerts and the robustness of the system, check the appendix.

False alarm handling

False alarms and false positives have been handled by adding the baseline vitals for each individual resident. These values have been defined using historical data over the past days. Most false alarms come from either pulse or motion (accelerometer) since there are sometimes that in a normal flow of vitals there is a peak with no apparent alert, which means that the sensor is failing or there is a system failure. These values must not be taken into consideration since they are not real alerts.

8. Conclusions and future work

System limitations

Overall, the alert system works coherently and allows to autonomously monitor the residents with notorious results, correctly identifying the emergency situations. However, the only issue could be the yellow alert coverage. Right now, the system is designed to trigger yellow alerts with very small nuances in the resident information, which can lead to some false positives. Even if we have handled false positives thoroughly, this could be fixed by making the rules stricter. This decision lies on the user or company that would use our system to monitor the residents and whether they want the yellow alerts to be very sensitive to small changes.

Future improvements

The system as it is works and performs as required. As of future improvements, some can be made. Firstly, a functionality to go back in time to observe past alerts could be included in the dashboard. Moreover, other LLM models could be used to improve the reasoning behind the alerts and get more accurate predictions. If GPU is available, context time windows could be fed into the agent providing past information that may be useful for the agent to decide the alert level and prevent false positives. Finally, the system could be tested with a bigger dataset with more residents to see how it performs.

Final conclusions

This project has helped us understand how agents work and how sometimes they are not strictly needed. The project started with the unique use of an agent as the final decisor and the team discovered the inefficiency of this and how this problem can be solved in a deterministic way. However, the use of agents for this problem still aids to solve ambiguous cases and determine the best actions to take, while keeping computational cost low. Overall, the system performs in a highly logical and precise manner and could be escalated to a real scenario. Although the system is evaluated on simulated data, the modular architecture allows direct adaptation to real sensor deployments with minimal changes. All the experiments performed are reproducible under fixed preprocessing pipelines, thresholds, and deterministic agent configurations. Finally, the system is designed as a decision-support tool and does not replace human judgment or medical diagnosis.

9. Appendices

Code structure

The code is done in a notebook which structure is separated in three phases as this whole report. Phase 1 covers data loading, exploratory analysis, preprocessing pipelines, baseline definition, and a heuristic alert system. Phase 2 focuses on supervised state prediction using an LSTM model and the integration of a hybrid LLM-based agent (Ollama and OpenAI) for the ambiguous cases. Phase 3 simulates real-time operation on a new day of data, applying the full pipeline and presenting results through an interactive Gradio dashboard.

Dashboard video

It can be accessed in the following link: [Dashboard Demo](#)

Additional alert cases descriptions

The additional alert cases are taken from the test dataset which contains the complete day 10/03/2025. A mix of alerts will be described to show robustness of the system.

1. RED alerts

The vast majority of red alerts are due to smoke events throughout the day for different residents.

On this day, a smoke event is detected starting at around 22:28:00 for the resident Enrique Posada Adadia. Although the resident is awake in bed with stable vital signs and no abnormal movement, the smoke concentration exceeds safe levels (≈ 55 ppm). The system correctly classifies this situation as a RED alert using the `definitely_red` rule and triggers an appropriate response by sending a caregiver. This alert persists consistently until approximately 22:40:00, indicating reliable detection of an environmental hazard.

At approximately 17:05:00, a severe smoke event is detected for the resident Graciela Zamorano Vergara. Despite the absence of abnormal movement or wearable alerts, the smoke concentration reaches extremely high levels (≈ 107 ppm) while the resident is detected as out of the room. The system correctly classifies this situation as a RED alert using the `definitely_red` rule and triggers an immediate escalation by sending a caregiver, which is an appropriate response to a critical environmental hazard. After this alert, we observe a fall event at around 17:40:00.

At 09:41:00, the resident Adela Arco-Llano is detected as SLEEPING while exhibiting high motion activity, as indicated by a strong accelerometer signal and active radar detection. Although no heuristic rule is triggered directly, this combination of signals is marked as ambiguous and escalated to the decision-making agent. The agent correctly interprets the situation as potentially dangerous, classifying it as a RED alert and recommending to send a caregiver to immediately check the resident's status.

2. ORANGE alerts

The majority of orange alerts are due to the spotting of residents outside of the room during night hours.

At 00:30:00, the resident Cristian Cazorla Cazorla is detected as out of the room during night hours. While physiological and environmental signals remain within normal ranges, this behavior is considered unusual and potentially risky given the time context. The system correctly triggers an ORANGE alert through the `definitely_orange` rule and escalates the situation by sending a caregiver, which is an appropriate preventive response for nighttime wandering.

At approximately 03:13:00, the resident Aníbal Bermudez Gual is detected as out of the room during night hours. Although physiological signals and motion levels are stable, this behavior is considered unusual given the time context. The system correctly applies the `definitely_orange` rule, generating an ORANGE alert and triggering the appropriate response by sending a caregiver to ensure the resident's safety.

At around 08:15:00, the resident Adrián de Fuentes is detected in an EATING state while exhibiting an extreme acceleration spike. Although vital signs and environmental conditions remain normal, the recorded movement intensity is highly inconsistent with both the resident state and radar information. The system correctly identifies this situation as abnormal using the `baseline_orange_movement` rule and triggers an ORANGE alert, appropriately escalating the case by sending a caregiver to assess a potential fall or unsafe movement.

3. YELLOW alerts

The yellow alerts represent those situations that do not require urgent care but they are rather unusual and should be taken care of with either a robot or caretaker.

At around 05:49:00, the resident Flora Anna Peinado Mir is detected in an EATING state during night hours. While all physiological and environmental signals remain within normal ranges, this behavior is unusual given the time context. The system correctly classifies the situation as a YELLOW alert using the `yellow_unusual_state_at_night` rule and opts to monitor the resident without immediate intervention.

At 13:09:00, the resident Adela Arco-Llano is detected as SLEEPING during midday hours. Although physiological signals and environmental conditions remain normal, the prolonged sleeping behavior is considered unusual for this time of day. The system correctly identifies this pattern using the `yellow_sleeping_late` rule, issuing a YELLOW alert and recommending sending a robot for routine, non intrusive assistance.

At around 12:53:00, the resident Flora Anna Peinado Mir is detected as SITTING_IN_CHAIR while presenting a moderately elevated heart rate relative to her baseline. Motion and environmental conditions remain stable, and no critical patterns are observed. The system correctly applies the `baseline_yellow_pulse` rule, classifying the situation as a YELLOW alert and opting to monitor the resident without immediate intervention.